SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of ArtificialIntelligence and Machine Learning

ASSESSMENT1 – Analytical Problem solving (5 QUESTIONS)

Course Code:	ITA05	**Course Title:**	**Computer Vision**
CO Assessed:	CO2	**Assessment:**	Assessment 1: Analytical Problem solving (5 Questions × 10 Marks)
Weightage:	35% of CIA	**Total Marks:**	
CO2 Statement:	***Apply image enhancement, filtering, a techniques to analyze images, detect ed boundaries, and solve real-world Com problems.***		
Student Name:	**DILLILOKESH D**	**Reg. No.:**	**192311097**
Section / Batch:	**SLOT A**	**Date:**	**21.10.2026**

Code of Conduct

*I (**DILLILOKESH D / 192311097**) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Dillilokesh D

QUESTIONS: 5 Total Marks: 50

Annexure A

Analytical Problems

Q1. A grayscale image contains the following pixel intensity values: 10, 10, 20, 20, 20, 30, 30, and 40. Construct the histogram by calculating the frequency of each intensity value. Calculate the cumulative distribution function (CDF) for the given image. Apply histogram equalization using the formula s = round((L − 1) × CDF), where L = 256. Determine the new intensity values and compare the contrast distribution before and after equalization.

Q2. A 3 × 3 image has the intensity values [[10, 20, 30], [20, 100, 20], [10, 20, 30]]. Apply a 3 × 3 averaging filter to calculate the new intensity value of the centre pixel. Replace the centre pixel with the calculated average while keeping the other pixels unchanged. Apply a median filter to the same neighbourhood and calculate the centre pixel value. Compare the results of the averaging and median filters and determine which filter better preserves image details in the presence of noise.

Q3. Consider a grayscale image with the intensity matrix [[50, 50, 50], [50, 100, 50], [50, 50, 50]]. Apply the Laplacian filter mask [[0, −1, 0], [−1, 4, −1], [0, −1, 0]] to calculate the response at the centre pixel. Use the calculated response to sharpen the centre pixel by adding it to the original intensity value. Determine the resulting centre pixel intensity after sharpening. Explain how the Laplacian filter enhances intensity transitions and fine details in the image.

Q4. A grayscale image contains the pixel intensity values [[40, 60, 120], [80, 150, 200], [30, 100, 180]]. Apply global thresholding with a threshold value of 100, assigning 1 to pixels greater than or equal to the threshold and 0 to the remaining pixels. Construct the resulting binary image. Calculate the number of foreground and background pixels after thresholding. Explain how the selected threshold separates objects from the background and identify one limitation of this method.

Q5. A grayscale image has the intensity matrix [[10, 10, 10], [10, 50, 90], [10, 50, 90]]. Apply the Sobel operator using the masks Gx = [[−1, 0, 1], [−2, 0, 2], [−1, 0, 1]] and Gy = [[−1, −2, −1], [0, 0, 0], [1, 2, 1]] to calculate the gradient components at the centre pixel. Calculate the gradient magnitude using $G = \sqrt{(Gx^2 + Gy^2)}$. Determine the dominant edge direction based on the calculated gradient components. Explain how the detected edge information can be used for boundary detection and image segmentation.

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Content Accuracy	30	Highly accurate, indepth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Faculty In-charge Signature: Date:	**Course Coordinator Program Director** Signature: Signature: Date: Date:



We have discussed the topic in the class

Answers

Question 1: Histogram Equalization

Given: Pixel intensities 10, 10, 20, 20, 20, 30, 30, 40. Total pixels N = 8 and number of grey levels L = 256.

1. Problem Identification

All eight pixels lie between 10 and 40, so the image uses only 30 of the 256 available grey levels (about 11.7%). It is therefore dark and low in contrast. The task is to build the histogram, compute the CDF, and use the CDF as a transfer function to spread the intensities over the full range.

2. Method and Formulae

- Probability: $p(r_k) = n_k / N$
- CDF: $CDF(r_k) = \Sigma\, p(r_j)$, summed from $j = 0$ to k
- Mapping: $s_k = \text{round}[(L - 1) \times CDF(r_k)] = \text{round}[255 \times CDF(r_k)]$

3. Step-by-Step Solution

Step 1 and 2: Histogram, probability, CDF and new level.

Intensity r	Frequency n	p = n/8	CDF	255 × CDF	New level s
10	2	0.250	0.250	63.75	64
20	3	0.375	0.625	159.375	159
30	2	0.250	0.875	223.125	223
40	1	0.125	1.000	255.00	255

Step 3: Replace each pixel. 10 → 64, 20 → 159, 30 → 223 and 40 → 255.

4. Final Result

Original pixels: 10, 10, 20, 20, 20, 30, 30, 40. Equalized pixels: **64, 64, 159, 159, 159, 223, 223, 255**.

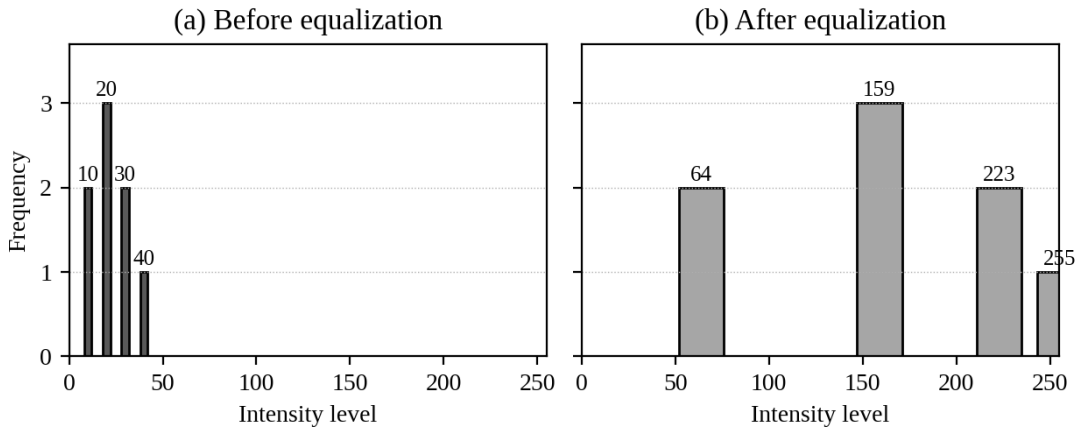


Figure 1: Histogram before and after equalization (grey levels written above each bar)

5. Analysis and Interpretation (Contrast Comparison)

Parameter	Before	After
Intensity range	10 to 40	64 to 255
Dynamic range (max − min)	30	191
Mean intensity	22.50	163.25
Standard deviation (contrast measure)	9.68	66.44
Gap between adjacent levels	10, 10, 10	95, 64, 32

Before equalization the histogram is crowded in the dark end, so details are hard to separate. After equalization the same pixels are spread across the range, the dynamic range grows by 191 / 30 ≈ 6.4 times and the standard deviation by about 6.9 times. The image becomes brighter and the differences between regions become clearly visible. Because there are only four distinct levels, no new levels are created; the existing ones are only moved apart.

6. Innovative Insight and Application

Linearity check. An ideally equalized image has a straight-line CDF, that is CDF = s / 255. The new levels give 64/255 = 0.251, 159/255 = 0.624 and 223/255 = 0.875, which almost exactly match the original CDF values 0.250, 0.625 and 0.875. This confirms that the mapping has flattened the cumulative distribution correctly. If the common

variant with CDF_min is used, the darkest level maps to 0 and the full 0 to 255 range is used. In practice, global equalization can over-amplify noise, so CLAHE (contrast-limited adaptive histogram equalization) is used for low-light surveillance frames and chest X-ray enhancement.

7. Tool Verification

```
import numpy as np
img = np.array([10,10,20,20,20,30,30,40])
v, n = np.unique(img, return_counts=True)
cdf = np.cumsum(n) / img.size
print(np.round(255 * cdf).astype(int))   # [ 64 159 223 255]
```

Question 2: Averaging and Median Filtering

Given: 3 × 3 image

10	20	30
20	100	20
10	20	30

1. Problem Identification

The centre value 100 is far larger than all eight neighbours (10 to 30). It behaves like an impulse (salt) noise pixel. The task is to compare a linear filter (mean) and a non-linear filter (median) and decide which one removes the spike while preserving the surrounding detail.

2. Method and Formulae

- Averaging filter: g = (1/9) × Σ of all nine pixels in the 3 × 3 window
- Median filter: sort the nine values and select the 5th (middle) value

3. Step-by-Step Solution

Averaging: Sum = 10 + 20 + 30 + 20 + 100 + 20 + 10 + 20 + 30 = 260. New centre = 260 / 9 = 28.89 ≈ **29**. The other pixels are unchanged.

10	20	30
20	29	20
10	20	30

Image after averaging

Median: Sorted values are 10, 10, 20, 20, **20**, 20, 30, 30, 100. The 5th value is 20, so the new centre = **20**.

10	20	30
20	20	20
10	20	30

Image after median filtering

4. Final Result

Filter	Centre value	Error from neighbourhood level (20)	Spike removed
Original	100	80	None
Averaging	29 (28.89)	9	About 89%
Median	20	0	100%

5. Analysis and Interpretation

The averaging filter only reduces the spike; the outlier is blended into the result, so the centre remains 9 units too bright and its error is shared with every neighbour (80 / 9 ≈ 8.9 added to each window that contains it). This also blurs edges. The median filter ignores the extreme value completely and returns a value that already exists in the neighbourhood, so no new grey levels or blur are introduced. **Conclusion: the median filter better preserves image details in the presence of noise.**

6. Innovative Insight and Application

Robustness view. The mean has a breakdown point of 0%, because a single extreme pixel can change it arbitrarily. The median has a breakdown point of 50%, so up to four corrupted pixels out of nine still leave the output correct. This is why median filters are preferred for dead-pixel correction in CCTV sensors, scratched document scans and

bit errors in satellite images. For Gaussian-type noise, however, the averaging (or Gaussian) filter is statistically more efficient, so the choice of filter must match the noise model.

7. Tool Verification

```
import numpy as np
p = np.array([[10,20,30],[20,100,20],[10,20,30]])
print(p.mean(), np.median(p))   # 28.888..., 20.0
```

Question 3: Laplacian Sharpening

Given: Image patch and Laplacian mask

50	50	50
50	100	50
50	50	50

Image patch

0	-1	0
-1	4	-1
0	-1	0

Laplacian mask

1. Problem Identification

The centre pixel (100) is brighter than its four-neighbours (50), representing a small bright detail on a flat background. The task is to measure this local intensity change with the second-derivative mask and use it to sharpen the pixel.

2. Method and Formulae

- Response: $R = \Sigma$ (mask value × corresponding pixel)
- Sharpening (centre coefficient positive): $g = f + R$, then clip to the range 0 to 255 for an 8-bit image

3. Step-by-Step Solution

Element-wise products (mask × patch):

0	-50	0
-50	400	-50
0	-50	0

R = 0 − 50 + 0 − 50 + 400 − 50 + 0 − 50 + 0 = 400 − 200 = **200**.

Sharpened value g = f + R = 100 + 200 = **300**.

4. Final Result

The Laplacian response at the centre is 200 and the raw sharpened intensity is 300. Since an 8-bit pixel cannot exceed 255, the displayed value is **255** after clipping.

5. Analysis and Interpretation

The Laplacian is a second-order derivative: it responds to the difference between a pixel and the average of its neighbours. Its coefficients sum to zero (0 − 1 + 0 − 1 + 4 − 1 + 0 − 1 + 0 = 0), so in a flat region the response is exactly 0 and nothing changes. Where intensity changes abruptly, such as at lines, points and edges, the response is large. Adding it back to the original exaggerates the transition (overshoot on the bright side, undershoot on the dark side), which makes fine detail look crisper. The disadvantage is that noise is also amplified.

6. Innovative Insight and Application

Response equals four times local contrast. Here the difference between centre and neighbour is 50, and R = 4 × 50 = 200. **Single-kernel equivalence:** f + R can be done in one pass with the kernel [0 −1 0; −1 5 −1; 0 −1 0], which gives 5(100) − 4(50) = 300, the same answer. **Controlled sharpening:** using g = f + c·R with c = 0.5 gives 100 + 100 = 200, which stays inside the valid range and avoids saturation. Such scaled Laplacian sharpening is used in OCR pre-processing, fingerprint ridge enhancement and medical radiographs, often after Gaussian smoothing (Laplacian of Gaussian) to limit noise.

7. Tool Verification

```
import numpy as np
f = np.array([[50,50,50],[50,100,50],[50,50,50]])
k = np.array([[0,-1,0],[-1,4,-1],[0,-1,0]])
R = (f * k).sum()                # 200
print(R, np.clip(f[1,1] + R, 0, 255))   # 200 255
```

Question 4: Global Thresholding

Given: Image [[40, 60, 120], [80, 150, 200], [30, 100, 180]] and threshold T = 100.

1. Problem Identification

The image has two intensity groups: a dark group (30, 40, 60, 80) and a bright group (100, 120, 150, 180, 200). The task is to separate bright objects (foreground) from the dark background using a single global threshold and to count the pixels in each class.

2. Method and Formulae

$g(x, y) = 1$ if $f(x, y) \geq T$, otherwise $g(x, y) = 0$, with $T = 100$.

3. Step-by-Step Solution

Row	Pixel comparison with T = 100	Binary row
1	40 → 0, 60 → 0, 120 → 1	0 0 1
2	80 → 0, 150 → 1, 200 → 1	0 1 1
3	30 → 0, 100 → 1 (equal to T), 180 → 1	0 1 1

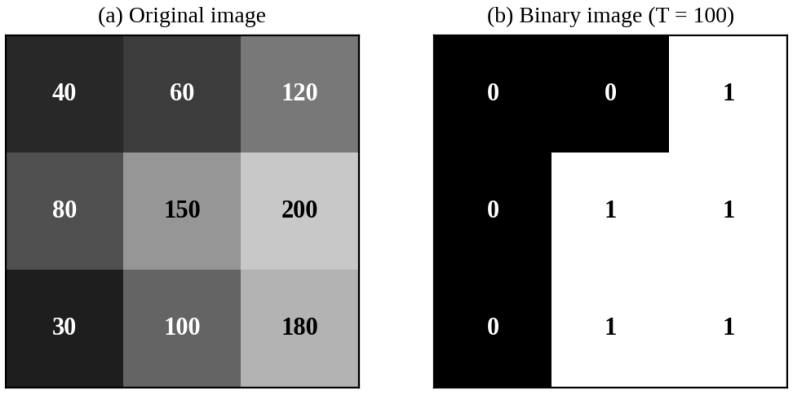


Figure 2: Original image and thresholded binary image

4. Final Result

Binary image = [[0, 0, 1], [0, 1, 1], [0, 1, 1]]. **Foreground pixels (1) = 5** (120, 150, 200, 100, 180), i.e. 55.6%. **Background pixels (0) = 4** (40, 60, 80, 30), i.e. 44.4%.

5. Analysis and Interpretation

The threshold sits in the gap between the two groups (highest dark value 80, lowest bright value 100). Every pixel at or above T is assigned to the object and every pixel below T to the background, which converts a grey-level problem into a simple two-class problem that is easy to count, measure and analyse. **Limitation:** a single fixed threshold ignores local variation. If lighting is uneven or noise is present, parts of the object may fall below T (or background above it), producing wrong segmentation.

6. Innovative Insight and Application

Threshold sensitivity study. The mean intensity of the image is 960 / 9 = 106.67. If this mean were used as the threshold, the pixel with value 100 would become background and the foreground would drop to 4. The table shows how the result depends on T.

Threshold T	Foreground count	Observation
80	6	80 is included, so a dark pixel becomes object
81 to 100	5	Stable range; same result as T = 100
106.67 (mean)	4	The boundary pixel 100 flips to background
150	3	Object shrinks; mid-tone pixels lost

The stable plateau from 81 to 100 shows that T = 100 is a safe, well-separated choice. For real images, automatic methods such as Otsu's method or adaptive (local) thresholding are used for document binarization, PCB defect detection and cell counting.

7. Tool Verification

```
import numpy as np
a = np.array([[40,60,120],[80,150,200],[30,100,180]])
b = (a >= 100).astype(int)
print(b); print(b.sum(), b.size - b.sum())   # 5 4
```

Question 5: Sobel Edge Detection

Given: Image patch

10	10	10
10	50	90
10	50	90

1. Problem Identification

Intensity rises from 10 to 50 to 90 from left to right and from top to bottom, so the centre pixel lies on an intensity transition. The task is to measure the horizontal and vertical gradients with the Sobel masks, find the edge strength and direction, and explain their use in boundary detection and segmentation.

2. Method and Formulae

- $G_x = \Sigma$ (Gx mask × patch) and $G_y = \Sigma$ (Gy mask × patch)

- Magnitude: $G = \sqrt{(Gx^2 + Gy^2)}$
- Gradient direction: $\theta = \tan^{-1}(Gy / Gx)$; the edge runs perpendicular to the gradient

3. Step-by-Step Solution

Row	Gx calculation	Gy calculation
Top	$(-1)(10) + 0(10) + 1(10) = 0$	$(-1)(10) + (-2)(10) + (-1)(10) = -40$
Middle	$(-2)(10) + 0(50) + 2(90) = 160$	0 (all weights are zero)
Bottom	$(-1)(10) + 0(50) + 1(90) = 80$	$1(10) + 2(50) + 1(90) = 200$
Total	**Gx = 240**	**Gy = 160**

Magnitude: $G = \sqrt{(240^2 + 160^2)} = \sqrt{(57{,}600 + 25{,}600)} = \sqrt{83{,}200} =$ **288.44**.

Direction: $\theta = \tan^{-1}(160 / 240) = \tan^{-1}(0.667) =$ **33.69°** (gradient direction, measured from the horizontal axis towards the downward direction). The edge direction is perpendicular, so it is 33.69° + 90° = **123.69°** (about 56.3° from the horizontal).

4. Final Result

Gx	Gy	Gradient magnitude G	Gradient direction	Edge direction
240	160	288.44	33.69°	123.69°

Since $|Gx| = 240$ is greater than $|Gy| = 160$, the horizontal intensity change dominates. The dominant edge is therefore **mostly vertical**, tilted slightly from the vertical axis.

5. Analysis and Interpretation (Boundary Detection and Segmentation)

Edge information is converted into boundaries and segments in four stages. First, the magnitude map G is thresholded so that only strong responses (such as 288.44) are kept as edge pixels. Second, the edges are thinned to one pixel width by non-maximum suppression along the gradient direction. Third, broken edge pixels are linked into continuous contours using the direction information and hysteresis thresholds (as in the Canny detector).

Finally, the closed contours separate the image into regions, which are labelled as objects. The gradient magnitude can also be used as the input surface for the watershed algorithm to produce region-based segmentation.

6. Innovative Insight and Application

Energy split. $G_x^2 / G^2 = 57{,}600 / 83{,}200 = 69.2\%$ and $G_y^2 / G^2 = 30.8\%$, which quantifies the vertical-edge tendency. **Relative strength.** The maximum possible Sobel response on an 8-bit image is 1020 per axis, giving $G_{max} = 1020\sqrt{2} \approx 1442.5$. The edge strength is $288.44 / 1442.5 \approx 20\%$, which classifies it as a moderate edge. **Built-in smoothing.** The weights (1, 2, 1) in the Sobel mask smooth the image while differentiating it, which makes it less noise sensitive than a plain difference mask. The same gradient information is applied in lane detection for driver assistance, PCB track inspection and tumour boundary outlining in medical images.

7. Tool Verification

```
import numpy as np
S  = np.array([[10,10,10],[10,50,90],[10,50,90]])
gx = np.array([[-1,0,1],[-2,0,2],[-1,0,1]])
gy = np.array([[-1,-2,-1],[0,0,0],[1,2,1]])
Gx, Gy = (S*gx).sum(), (S*gy).sum()      # 240, 160
print(np.hypot(Gx, Gy), np.degrees(np.arctan2(Gy, Gx)))   # 288.44, 33.69
```